

Library Book Inventory Manager

Technical Documentation & Source Code Overview

Domain:	Python Programming
Document Type:	Project Documentation
Date:	June 16, 2026
Status:	Completed Submission

1. Executive Summary

The **Library Book Inventory Manager** is a command-line interface (CLI) application built using modern Python 3 object-oriented design principles. Developed as part of the **SyntecxHub Python Programming Internship** requirement, this application serves as a robust prototype for managed system utilities handling relational records in persistent JSON flat-file data storage.

The application encapsulates data attributes and behaviors using modular classes (`Book` and `Library`), ensuring separation of concerns, high code maintainability, and clean standard input/output streaming.

2. Architecture and Data Structures

The backend infrastructure utilizes two primary entities structured to support full CRUD (Create, Read, Update, Delete) transactional routines:

2.1 The Book Class

Represents an individual physical book asset with unique metadata bindings. It contains structural procedures to serialize its state into native Python dictionaries compatible with standard serializer codecs.

- `book_id` (String): A unique text identifier/SKU key for lookup mapping.
- `title` (String): Title heading of the publication.
- `author` (String): Author entity attribution text.
- `issued` (Boolean): Binary availability state flag (True = Checked out, False = On Shelf).

2.2 The Library Class

Functions as the central controller orchestration engine. It acts as the business logic facade layer mapping runtime memory data state structures directly to the physical storage device via standard `json` file mapping operations.

Persistence Mechanism:

All data transfers utilize `library.json` as a zero-dependency database layer. Reading routines are handled gracefully with defensive exception catches to handle empty or malformed structures.

3. Feature Breakdown Matrix

Feature Name	Functional Description	Data/State Impact
Add Book	Appends a brand-new asset ledger entry following ID duplication validations.	Appends to dictionary; Triggers <code>save_data()</code> .
Search Book	Performs a case-insensitive lookup using substring matching filters across titles/authors.	Read-Only execution.
Issue Book	Toggles state from available to borrowed after checking eligibility conditions.	Updates <code>issued</code> flag to True.
Return Book	Restores asset inventory state to baseline availability upon physical verification.	Updates <code>issued</code> flag to False.
Display All	Renders ASCII structured view tables formatted properly with padded structural tabular widths.	Read-Only execution.
Delete Book	Permanently ejects records corresponding to user-specified keys.	Removes from map dictionary tree entirely.

4. Complete Source Code Implementation

Below is the exact production-ready implementation codebase of the `library_book_inventory_manager.py` system script compiled for the SyntecxHub evaluation:

```
import json
import os

class Book:
    def __init__(self, book_id, title, author, issued=False):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.issued = issued

    def to_dict(self):
        return {
            "book_id": self.book_id,
            "title": self.title,
            "author": self.author,
            "issued": self.issued
        }

class Library:
    def __init__(self, filename="library.json"):
        self.filename = filename
        self.books = {}
        self.load_data()

    def load_data(self):
        if os.path.exists(self.filename):
            try:
                with open(self.filename, "r") as file:
                    data = json.load(file)
                    for book in data:
                        self.books[book["book_id"]] = Book(
                            book["book_id"],
                            book["title"],
                            book["author"],
                            book["issued"]
                        )
            except json.JSONDecodeError:
                self.books = {}
```

```

def save_data(self):
    with open(self.filename, "w") as file:
        json.dump(
            [book.to_dict() for book in self.books.values()],
            file,
            indent=4
        )

def add_book(self):
    book_id = input("Enter Book ID: ").strip()

    if book_id in self.books:
        print("Book ID already exists!")
        return

    title = input("Enter Book Title: ").strip()
    author = input("Enter Author Name: ").strip()

    self.books[book_id] = Book(book_id, title, author)
    self.save_data()

    print("Book added successfully!")

def search_book(self):
    keyword = input("Enter Title or Author to Search: ").lower()

    found = False

    for book in self.books.values():
        if keyword in book.title.lower() or keyword in book.author.lower():
            status = "Issued" if book.issued else "Available"

            print("-" * 50)
            print(f"ID      : {book.book_id}")
            print(f"Title   : {book.title}")
            print(f"Author  : {book.author}")
            print(f"Status  : {status}")
            found = True

    if not found:
        print("No matching books found!")

def issue_book(self):
    book_id = input("Enter Book ID to Issue: ").strip()

    if book_id not in self.books:

```

```

        print("Book not found!")
        return

    if self.books[book_id].issued:
        print("Book is already issued!")
        return

    self.books[book_id].issued = True
    self.save_data()

    print("Book issued successfully!")

def return_book(self):
    book_id = input("Enter Book ID to Return: ").strip()

    if book_id not in self.books:
        print("Book not found!")
        return

    if not self.books[book_id].issued:
        print("Book is already available!")
        return

    self.books[book_id].issued = False
    self.save_data()

    print("Book returned successfully!")

def display_books(self):
    if not self.books:
        print("No books available!")
        return

    print("\n" + "=" * 75)
    print(f"{'ID':<10}{'TITLE':<30}{'AUTHOR':<20}{'STATUS':<15}")
    print("=" * 75)

    for book in self.books.values():
        status = "Issued" if book.issued else "Available"

        print(
            f"{book.book_id:<10}"
            f"{book.title:<30}"
            f"{book.author:<20}"
            f"{status:<15}"
        )

```

```

def generate_report(self):
    total_books = len(self.books)
    issued_books = sum(
        1 for book in self.books.values()
        if book.issued
    )

    available_books = total_books - issued_books

    print("\n===== LIBRARY REPORT =====")
    print(f"Total Books      : {total_books}")
    print(f"Issued Books       : {issued_books}")
    print(f"Available Books    : {available_books}")

def delete_book(self):
    book_id = input("Enter Book ID to Delete: ").strip()

    if book_id not in self.books:
        print("Book not found!")
        return

    del self.books[book_id]
    self.save_data()

    print("Book deleted successfully!")

def main():
    library = Library()

    while True:
        print("\n" + "=" * 40)
        print("LIBRARY BOOK INVENTORY MANAGER")
        print("=" * 40)
        print("1. Add Book")
        print("2. Search Book")
        print("3. Issue Book")
        print("4. Return Book")
        print("5. Display All Books")
        print("6. Generate Report")
        print("7. Delete Book")
        print("8. Exit")

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            library.add_book()

```

```
elif choice == "2":
    library.search_book()

elif choice == "3":
    library.issue_book()

elif choice == "4":
    library.return_book()

elif choice == "5":
    library.display_books()

elif choice == "6":
    library.generate_report()

elif choice == "7":
    library.delete_book()

elif choice == "8":
    print("Thank you for using Library Book Inventory Manager!")
    break

else:
    print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()
```

5. Conclusion

The application adheres fully to professional modular Python standard coding criteria. It establishes safe state mutations across multiple CRUD pathways while providing user feedback loops, making it an excellent demonstration of foundational engineering milestones under the **SyntecxHub Internship Framework**.